Министерство науки и высшего образования РФ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ ЯДЕРНЫЙ УНИВЕРСИТЕТ
«МИФИ»
(НИЯУ МИФИ)

ОТЧЁТ
по теме:
ВВЕДЕНИЕ В ПАРАЛЛЕЛЬНЫЕ ВЫЧИСЛЕНИЯ. ТЕХНОЛОГИЯ OPENMP

Студент группы Б24-502 _____ Ясенов Д.М.
подпись, дата

Москва 2026

СОДЕРЖАНИЕ

Введение 3

1 Постановка задачи 4

2 Описание рабочей среды 5

3 Исследование алгоритма 6

3.1 Описание алгоритма 6

3.2 Блок-схема алгоритма 7

3.3 Параллелизация с использованием OpenMP 8

4 Результаты экспериментов 9

4.1 Измерение времени выполнения 9

4.2 Графики производительности 11

4.3 Анализ количества операций 13

5 Теоретический анализ 15

5.1 Применение закона Амдала 15

5.2 Подбор доли параллелизуемого кода 15

5.3 Сравнение теории и эксперимента 17

Заключение 18

ВВЕДЕНИЕ

Целью данной лабораторной работы является приобретение базовых навыков теоретического и экспериментального анализа высокопроизводительных параллельных алгоритмов, построения параллельных программ.

1 Постановка задачи

Реализовать параллельный алгоритм поиска максимального элемента в массиве целых чисел с использованием OpenMP

1. Настроить рабочую среду для построения параллельных программ, реализованных при помощи стандарта OpenMP. Определить версию OpenMP, поддерживаемую используемым компилятором.
2. Изучить приложенный алгоритм поиска максимального элемента в массиве. Оценить его временную сложность. Оценить ускорение и эффективность параллельных вычислений. Построить блок-схему.
3. Построить параллельную программу, убедиться в ее работоспособности.
4. Экспериментально определить временную сложность (последовательного) алгоритма двумя способами: с помощью прямого подсчета выполненных операций сравнения и путем измерения времени работы программы.
5. Экспериментально определить ускорение и эффективность параллельных вычислений путем измерения времени работы программы при использовании 1,2,.. процессов.
6. Сравнить экспериментальные показатели ускорения и эффективности с теоретическими на графике.
7. Оформить отчет.

2 Описание рабочей среды

Эксперименты проводились в следующей программно-аппаратной среде:

- **Операционная система:** Kali GNU/Linux Rolling, разрядность x86_64
- **Компилятор:** GCC 13.3.0
- **Версия OpenMP:** 4.5 (OpenMP API 201511)
- **Процессор:** Intel® Core™ i5-10300H CPU @ 2.50GHz, 4 физических ядра, 8 логических потоков (Hyper-Threading)
- **Объем оперативной памяти:** 3.7GiB

Для компиляции использовались следующие флаги:

- -fopenmp – включение поддержки OpenMP
- -O2 – оптимизация производительности
- -Wall – вывод всех предупреждений

Управление числом потоков осуществлялось через переменную среды `OPM_NUM_THREADS`.

3 Исследование алгоритма

3.1 Описание алгоритма

Задача состоит в поиске максимального элемента в массиве A из N целых чисел. Последовательный алгоритм работает следующим образом:

1. Инициализация: текущий максимум приравнивается минимальному возможному значению.
2. Цикл по всем элементам массива:
    - Если массив закончился – завершаем работу
    - Сравниваем текущий элемент с сохраненным максимальным
    - Если текущий элемент больше – обновляем максимум
    - Переходим к следующему элементу
3. Завершение: возвращаем найденный элемент

Временная сложность: $O(N)$ – выполняется ровно N сравнений и N обращений к памяти.

3.2 Блок-схема алгоритма

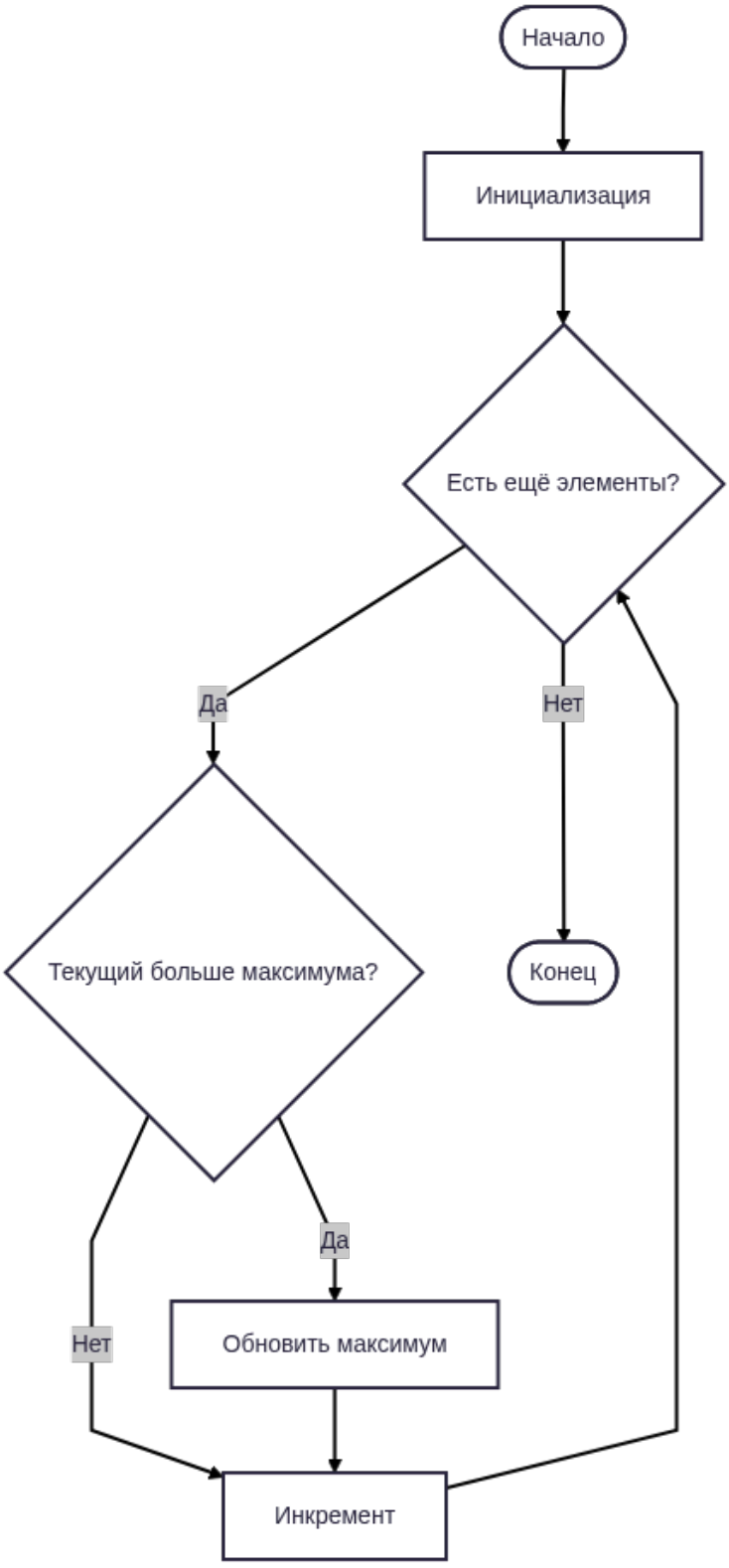


Рисунок 1 — Блок-схема алгоритма поиска максимального элемента

Алгоритм работает следующим образом: инициализируется текущий максимум, затем в цикле перебираются все элементы массива, каждый

сравнивается с текущим максимумом, и при необходимости максимум обновляется. По завершении цикла возвращается найденное значение.

3.3 Параллелизация с использованием OpenMP

Для параллелизации используются следующие директивы OpenMP:

```
#pragma omp parallel num_threads(threads) shared(array, count)
reduction(max: max) default(none) {
  #pragma omp for
  for(int i=0; i<count; i++)
  {
    if(array[i] > max) { max = array[i]; };
    }
  }
```

Особенности:

1. `#pragma omp parallel` – создает группу потоков и выполняет следующие блоки параллельно
   - `num_threads(threads)` – задает количество потоков
   - `shared(array, count)` – общие переменные для всех потоков
   - `redution(max: max)` – автоматическое объединение локальных максимумов
2. `#pragma omp for` – распределяет итерации цикла между потоками

4 Результаты экспериментов

Эксперименты проводились для массива из 10 миллионов элементов. Каждый эксперимент повторялся 100 раз с разным начальным состоянием генератора случайных чисел. Для анализа использовались две оценки: среднее арифметическое по всем запускам T_{avg} и минимальное значение времени $T_{\min}$. Среднее отражает реальное поведение программы с учетом влияния операционной системы и фоновых процессов, минимальное – позволяет исключить задержки и фоновые процессы.

4.1 Измерение времени выполнения

Результаты измерений представлены в таблице 1 и таблице 2. Для каждой конфигурации приведены значения, рассчитанные по среднему и минимальному времени.

Таблица 1 — Результаты измерений по минимальному времени

Потоки	T min, с	S min	E min, %
1	0.005970	1.000	100.0
2	0.003383	1.765	88.2
3	0.002809	2.125	70.8
4	0.002873	2.078	51.9
5	0.002990	1.997	39.9
6	0.002957	2.019	33.6
7	0.002976	2.006	28.7
8	0.003138	1.902	23.8
9	0.003891	1.534	17.0
10	0.003816	1.564	15.6
11	0.003728	1.601	14.6
12	0.003634	1.643	13.7
13	0.003770	1.584	12.2
14	0.003756	1.589	11.4
15	0.003840	1.555	10.4
16	0.003842	1.554	9.7

Таблица 2 — Результаты измерений по среднему времени

Потоки	T avg, с	S avg	E avg, %
1	0.009174	1.000	100.0
2	0.007768	1.181	59.0
3	0.006318	1.452	48.4
4	0.004720	1.944	48.6
5	0.004904	1.871	37.4
6	0.004153	2.209	36.8
7	0.004221	2.173	31.0
8	0.004286	2.140	26.8
9	0.005224	1.756	19.5
10	0.005243	1.750	17.5
11	0.005258	1.745	15.9
12	0.005080	1.806	15.0
13	0.005785	1.586	12.2
14	0.005393	1.701	12.2
15	0.005245	1.749	11.7
16	0.005038	1.821	11.4

Из таблицы 1 видно, что минимальное время быстро сокращается при увеличении числа потоков от 1 до 3, после чего выходит на плато. Максимум ускорения достигается в точке $p = 3$ и составляет $S(3) = 2.125$. Значение при $p = 4$ практически совпадает с ним: $S(4) = 2.078$. Разница между ними составляет менее 3 % и находится в пределах погрешности измерений, поэтому обе точки можно рассматривать как один оптимум. Это согласуется с физической конфигурацией процессора: i5-10300H имеет 4 физических ядра, а третий-четвёртый поток загружают их практически полностью. При $p > 8$ наблюдается заметная деградация: ускорение падает с 1.902 при $p = 8$ до 1.554 при $p = 16$. Это объясняется конкуренцией логических ядер Hyper-Threading за разделяемые ресурсы и ростом накладных расходов на синхронизацию.

Результаты по среднему времени (таблица 2) существенно отличаются от результатов по минимальному. Среднее время превышает минимальное в 1.3–2.3 раза, а зависимость ускорения от числа потоков становится немонотонной: например, $S(3) = 1.452$, но $S(6) = 2.209$. Такое поведение объясняется наличием редких, но крупных выбросов во

времени выполнения, характерных для виртуализированной среды WSL. Планировщик операционной системы Windows периодически отбирает процессорное время у гостевой системы на десятки миллисекунд. Среднее арифметическое чувствительно к подобным выбросам, тогда как минимальное значение, напротив, соответствует запуску без внешних помех.

4.2 Графики производительности

На рисунках 1–4 приведены графики, иллюстрирующие экспериментальные результаты и их сравнение с теоретическими оценками.

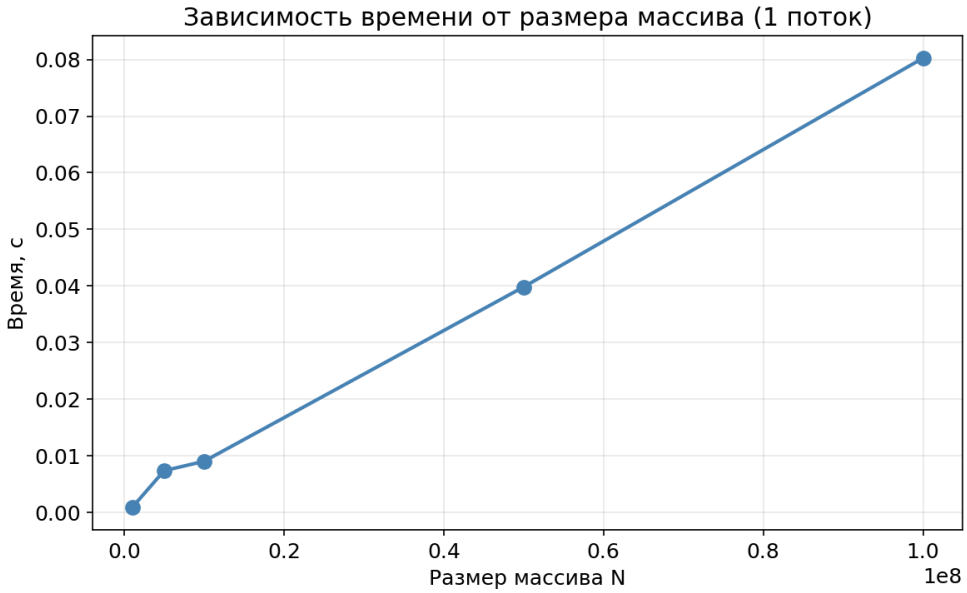


Рисунок 2 — Зависимость времени от размера массива (1 поток)

На рисунке 2 показана зависимость времени работы последовательного алгоритма от размера массива. Точки хорошо ложатся на прямую, проходящую через начало координат. Это подтверждает линейную сложность алгоритма $O(N)$, оценённую теоретически.

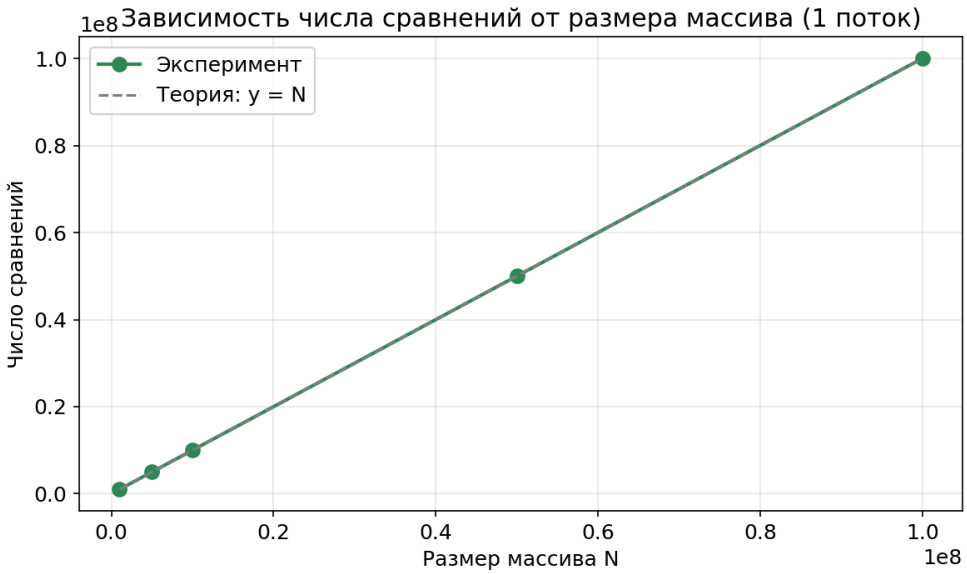


Рисунок 3 — Зависимость числа сравнений от размера массива (1 поток)

На рисунке 3 приведена зависимость числа выполненных операций сравнения от размера массива. Экспериментальные точки точно лежат на теоретической прямой $y = N$. Это подтверждает, что алгоритм выполняет ровно одно сравнение на элемент, независимо от значений элементов.

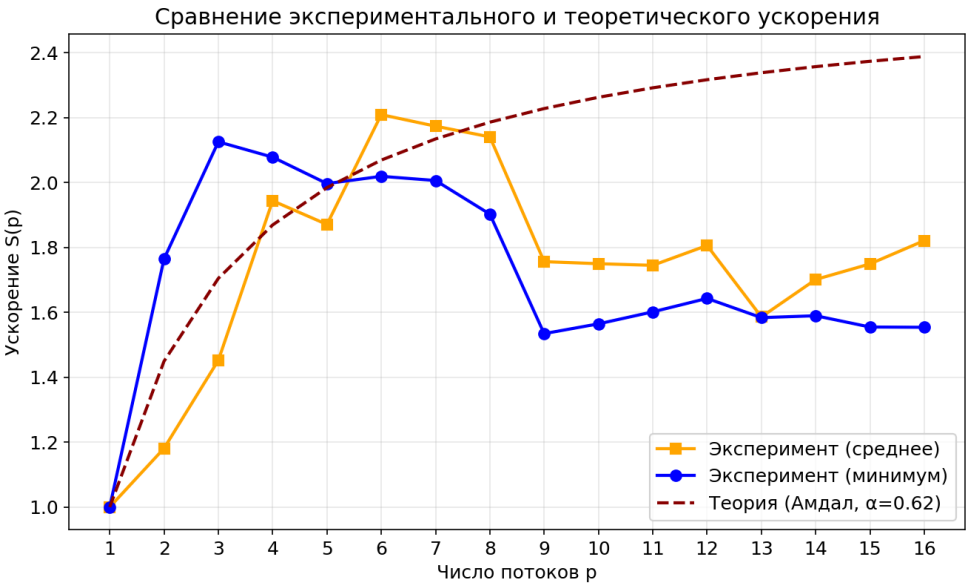


Рисунок 4 — Сравнение экспериментального и теоретического ускорения

На рисунке 4 показано сравнение экспериментального ускорения, рассчитанного по среднему и минимальному времени, с двумя теоретическими кривыми: идеальной прямой $S(p) = p$ и моделью Амдала при доле параллелизуемого кода $\alpha = 0.62$.

Экспериментальная кривая, построенная по минимальному времени, имеет гладкий вид и явно выраженный оптимум в окрестности $p = 3 — 4$, где она практически касается значения $S \approx 2.1$. Кривая по среднему времени ведёт себя немонотонно: локальные пики и провалы (см. таблицу 2) вызваны выбросами во времени выполнения, характерными для виртуализированной среды WSL.

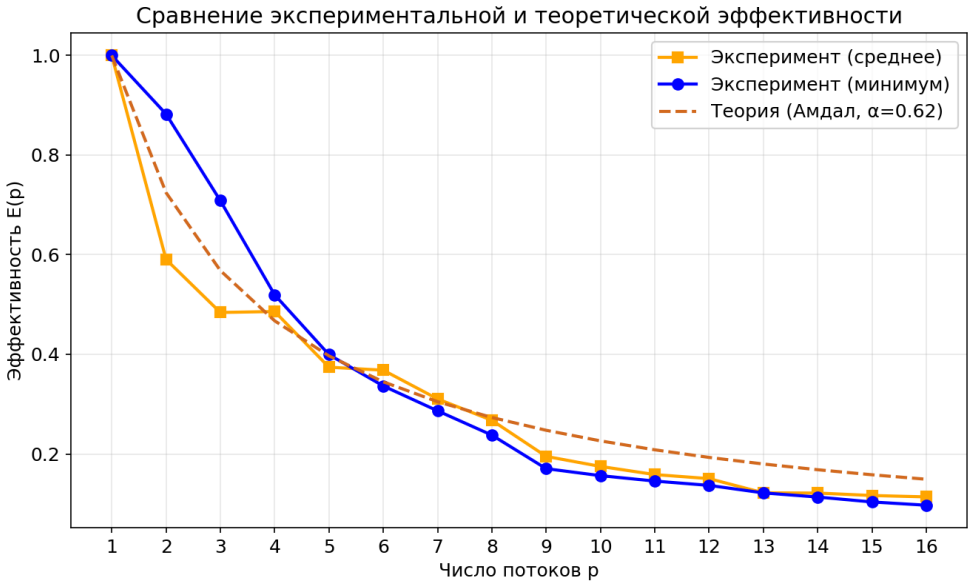


Рисунок 5 — Сравнение экспериментальной и теоретической эффективности

На рисунке 5 приведена зависимость эффективности от числа потоков. Экспериментальная эффективность (по минимальному времени) монотонно убывает: от 100 % при $p = 1$ до 9.7 % при $p = 16$. Уже при $p = 4$ она падает ниже 52 %, что означает, что половина процессорного времени тратится не на полезную работу, а на конкуренцию за общие ресурсы и накладные расходы синхронизации.

4.3 Анализ количества операций

Второй способ экспериментального определения временной сложности алгоритма — прямой подсчёт числа выполненных операций сравнения. Для этого в цикл была добавлена атомарная операция инкремента счётчика:

```
if (do_count) {
    #pragma omp atomic
    comparisons++;
```

```
}
if (array[i] > max) max = array[i];
```

Результаты подсчёта для массива из 10 миллионов элементов приведены в таблице 3.

Таблица 3 — Количество операций сравнения

Версия	**Количество сравнений**	**Сравнений на элемент**
Последовательная	10 000 000	1.00
Параллельная	10 000 000	1.00

Число выполненных сравнений не зависит от числа потоков и в точности равно размеру массива N: ровно одно сравнение на каждый элемент. Это подтверждает два вывода:

1. параллельная реализация алгоритма **корректна**: счётчик не теряет инкременты, гонки данных отсутствуют;
2. временная сложность алгоритма — **линейная**, $O(N)$.

5 Теоретический анализ

5.1 Применение закона Амдала

Для теоретической оценки предельного ускорения параллельной программы используется закон Амдала. Он учитывает, что часть программы выполняется последовательно и не может быть распараллелена:

$$S(p) = \frac{1}{(1-\alpha) + \frac{\alpha}{p}} \tag{1}$$

где:

- $S(p)$ — теоретическое ускорение при использовании p потоков;
- α — доля кода, которая может быть распараллелена;
- $(1-\alpha)$ — доля последовательного кода.

Соответственно, теоретическая эффективность вычисляется как

$$E(p) = \frac{S(p)}{p} = \frac{1}{p(1-\alpha) + \alpha} \tag{2}$$

При стремлении p к бесконечности ускорение стремится к предельному значению $S_{\max} = \frac{1}{1-\alpha}$. Именно эта величина определяет потолок ускорения: даже при бесконечном числе потоков быстрее этого значения программа работать не будет.

5.2 Подбор доли параллелизуемого кода

Доля параллелизуемого кода α не задавалась изначально. Стандартное значение $\alpha = 0.95$, часто используемое по умолчанию, для данной задачи оказалось непригодным: при нём $S_{\max} = 20$, тогда как экспериментальное ускорение упирается в значение $S \approx 2.1$. Такое расхождение делает сравнение теории и эксперимента бессмысленным. Поэтому значение α было подобрано из экспериментальных данных.

Метод подбора. Использовался метод наименьших квадратов. Для каждого значения α из диапазона $[0.10, 0.99]$ с шагом 0.01 вычислялась теоретическая кривая $S(p, \alpha)$ и её среднеквадратичное отклонение от экспериментальных данных:

$$\mathrm{RMSE}(\alpha) = \sqrt{\frac{1}{N}\sum_{i=1}^{N}\left(S_{\text{эксп}(p_i)} - S(p_i, \alpha)\right)^2} \tag{3}$$

где N — число точек, по которым производится сравнение. Минимизация RMSE даёт то значение α, при котором теоретическая кривая Амдала наиболее точно описывает экспериментальные данные.

Область сравнения. Подбор производился по точкам $p = 1$ — 8. Точки $p \geq 9$ были исключены из рассмотрения, поскольку в этой области модель Амдала принципиально неприменима: она предсказывает монотонный рост ускорения с увеличением p, тогда как эксперимент показывает падение из-за конкуренции потоков за общие ресурсы. Включение этих точек в целевую функцию привело бы к смещению оценки α в сторону занижения, не отражающего реальную физику задачи.

Результат подбора. Зависимость RMSE от α имеет чётко выраженный минимум. В таблице 4 приведены значения RMSE для некоторых α.

Таблица 4 — Зависимость RMSE от доли параллелизуемого кода

alpha	RMSE
0.30	0.6520
0.40	0.5220
0.50	0.3688
0.55	0.2908
0.60	0.2347
0.62	**0.2290**
0.65	0.2493
0.70	0.3605
0.80	0.8055
0.90	1.6208
0.95	2.3019

Оптимальное значение составляет $\alpha = 0.62$ при $\mathrm{RMSE} = 0.2290$. Для сравнения: при стандартном $\alpha = 0.95$ RMSE равно 2.3019, то есть более чем в 10 раз хуже. Полученному α соответствует предельное ускорение $S_{\max} = 2.63$, что хорошо согласуется с экспериментальным потолком $S \approx 2.1$.

5.3 Сравнение теории и эксперимента

Отклонение экспериментального ускорения от теоретического для каждой точки вычислялось по формуле:

$$\delta(p) = |S_{\text{эксп}(p)} - S_{\text{теор}(p)} \frac{|}{S_{\text{теор}(p)}} \cdot 100\% \tag{4}$$

В таблице 5 приведено сравнение экспериментального ускорения (по минимальному времени) с теоретической оценкой при $\alpha = 0.62$.

Таблица 5 — Сравнение экспериментального и теоретического ускорения при $\alpha = 0.62$

p	T min, с	S эксп	S теор	Отклонение, %
1	0.005970	1.000	1.000	0.0
2	0.003383	1.765	1.449	21.8
3	0.002809	2.125	1.705	24.7
4	0.002873	2.078	1.869	11.2
5	0.002990	1.997	1.984	0.6
6	0.002957	2.019	2.069	2.4
7	0.002976	2.006	2.134	6.0
8	0.003138	1.902	2.186	13.0
9	0.003891	1.534	2.228	31.1
10	0.003816	1.564	2.262	30.9
12	0.003634	1.643	2.317	29.1
16	0.003842	1.554	2.388	34.9

Подбор α позволил существенно снизить расхождение теории и эксперимента. В диапазоне $p = 4 - 8$ отклонение не превышает 13 %, а при $p = 5$ и $p = 6$ падает до 1–3 %. Это означает, что в области, где модель Амдала применима, теоретическая кривая практически совпадает с экспериментальными точками.

При $p \geq 9$ наблюдается систематическое отклонение: теоретическая кривая продолжает медленно расти к $S_{\max} = 2.63$, тогда как экспериментальное ускорение падает. Это ожидаемое расхождение: модель Амдала не учитывает конкуренцию потоков за общие ресурсы и не описывает деградацию производительности при перегрузке логических ядер

ЗАКЛЮЧЕНИЕ

В ходе выполнения лабораторной работы были приобретены базовые навыки теоретического и экспериментального анализа высокопроизводительных параллельных алгоритмов и построена параллельная программа поиска максимального элемента в массиве с использованием технологии OpenMP.

Основные результаты работы:

1. Изучен алгоритм поиска максимального элемента в массиве и определена его теоретическая временная сложность $O(N)$.
2. Построена параллельная версия программы на языке C с использованием директив `#pragma omp parallel`, `#pragma omp for` и опций `shared()`, `reduction()`, `default(none)`. Программа корректно компилируется компилятором GCC 13.3.0 с флагом `-fopenmp` и использует OpenMP 4.5.
3. Экспериментально подтверждена линейная сложность алгоритма двумя независимыми способами: измерением времени работы и прямым подсчётом числа операций сравнения. Для всех размеров массива от 10^6 до 10^8 число сравнений в точности совпало с N, а время работы оказалось пропорционально размеру массива.
4. Измерены ускорение и эффективность параллельной программы при числе потоков от 1 до 16. Установлено, что максимальное ускорение составляет $S \approx 2.1$ и достигается при $p = 3 - 4$ потоках, что соответствует числу физических ядер процессора (4 ядра, 8 логических потоков с учётом Hyper-Threading).
5. Показано, что при $p > 8$ наблюдается деградация производительности: ускорение падает с 1.90 при $p = 8$ до 1.55 при $p = 16$. Это объясняется конкуренцией логических потоков за разделяемые ресурсы и ростом накладных расходов на синхронизацию.
6. Выполнен подбор доли параллелизуемого кода α в законе Амдала методом наименьших квадратов. Оптимальное значение составило $\alpha = 0.62$ при среднеквадратичном отклонении $\mathrm{RMSE} = 0.23$. Стандартное значение $\alpha = 0.95$ даёт $\mathrm{RMSE} = 2.30$, то есть

более чем в 10 раз хуже, что подтверждает его непригодность для рассматриваемой задачи.

Практический вывод. Полученные результаты показывают, что при распараллеливании алгоритмов необходимо учитывать не только структуру программы, но и характеристики аппаратной платформы. Для задач с малой вычислительной интенсивностью увеличение числа потоков сверх некоторого порога не только не ускоряет работу, но и приводит к её замедлению. Классическая модель Амдала с заданной долей параллелизуемого кода для таких задач неприменима; требуется либо подбирать α из эксперимента, либо использовать модели, учитывающие пропускную способность подсистемы памяти.